



CS6482 Deep Reinforcement Learning

Assignment 1: Convolutional Neural Networks (CNNs)

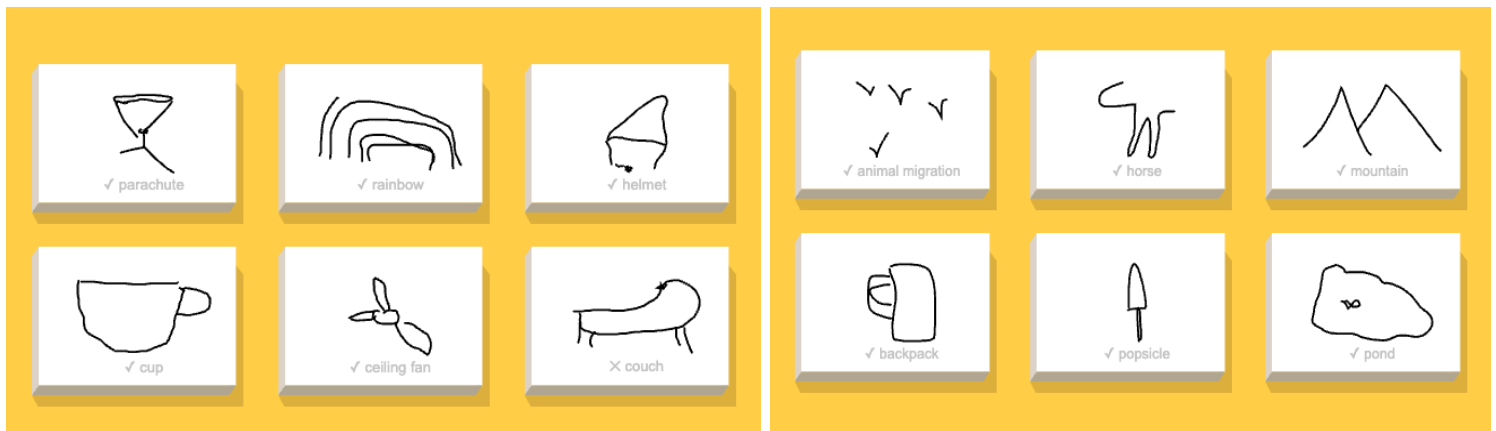
Jai Parakh [25165933]

Simran Kukreja [25141104]

MSc in Artificial Intelligence and Machine Learning

Sem 2 AY 25/26

Submitted to: Prof. JJ Collins



By Jai

By Simran

*We dedicate this report to our masterpieces we created while playing
“[QuickDraw by Google](#)”.
We look forward to improving our doodling skills. 😊*

Table of Contents

1. Introduction	4
2. Dataset	5
2.1 Data Loading	5
2.2 Data Visualisation	6
2.3 Data Pre Processing	8
2.4 Data Split	8
3. Network Structure	9
3.1 Network Architecture	9
4. Loss function	11
5. Optimiser	12
6. Experiments	12
6.1 Data Preprocessing	12
6.1.1 Input Channel/Size	12
6.1.2 Random Affine	14
6.2 Layer Freezing	15
6.3 Learning Rate	16
6.4 Dropout	17
6.5 Training from Scratch	18
7. Results	18
7.1 Pretrained Model	18
7.2 Model Trained from Scratch	20
8. Suggested Improvements	21
9. Related Work	22
10. Statement of Work	22
11. AI Prompts Used	23
12. Resources	23

1. Introduction

This report presents our results of applying the ConvNeXt [1] model to perform image classification on the QuickDraw dataset by Google [4]. ConvNeXt, proposed by Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrel, Saining Xie is a pure ConvNet model [3]. This model is designed to be simple and efficient in design and is said to have better accuracy and performance than Vision Transformers [7]. The main idea for ConvNeXt was to first understand why Vision Transformers perform better than CNNs and then see if those design principles can be added to CNNs. The preprocessed version of the dataset is used where the images were already resized and centered and converted to binary numpy files. We have explored two different approaches for this classification problem, where we have compared a pretrained version of the model with the one we trained from scratch. Visualisation is done to understand and plot some of the key features of the dataset, including correlation between different classes, average stroke distribution and sparsity distribution. During data transformation, we have resized the images to 56x56 and normalised for faster convergence. To improve the model after every epoch we have used the training, validation and test set approach, and used Cross-Entropy loss to generate the loss curve. ConvNeXt uses GeLU as the default activation function, and we have stuck to that for this project. AdamW is used as the optimiser based on the experiments performed while training ConvNeXt. We performed a number of experiments to improve hyperparameters, tackle overfitting which was caused by the pretrained model and compare test accuracies between pretrained and scratch versions of ConvNeXt. Experimental results show that a model trained from scratch gives much better results and avoids overfitting than the pretrained one for the same dataset.

2. Dataset

Our aim was to use a large image dataset as ConvNeXt is a simple CNN model. QuickDraw by Google is a significantly large dataset with 50 million doodled drawings contributed by over 15 million users for 345 different classes [4].

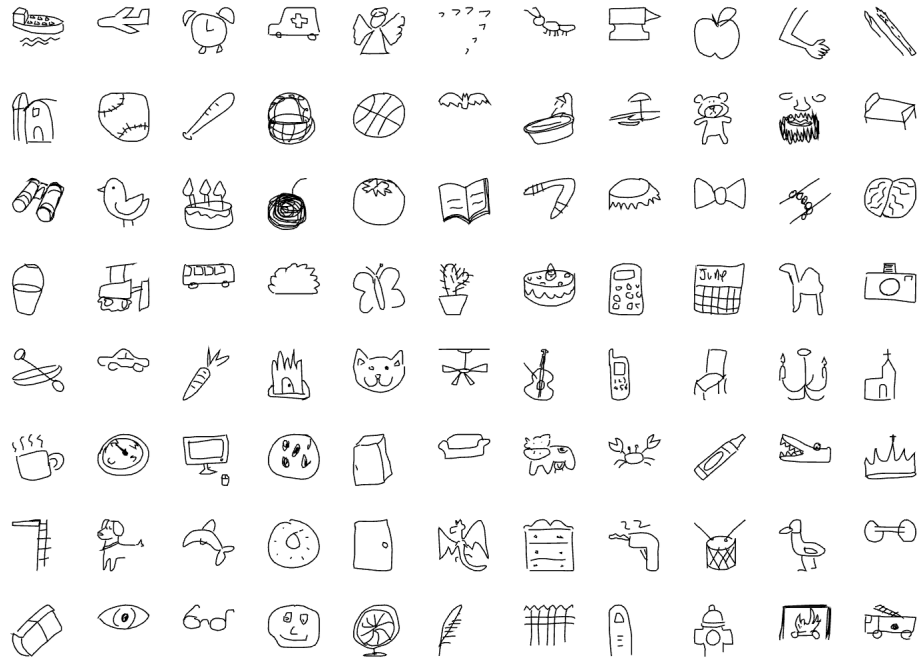


Figure 1: QuickDraw dataset by Google, doodles drawn by users.

Google has released both raw and preprocessed versions of the data. Raw data contains timestamped vectors, the drawing asked to be drawn and the country of the user. Whereas, preprocessed data contains the raw data preprocessed and split into different files and formats to make it easier and faster to download and use.

We have used the preprocessed version of the data, specifically the numpy version where the drawings have been rendered into a 28x28 grayscale bitmap in numpy (.npy) format. Each .npy file is based on a category, that means we have 345 numpy files with each file containing drawings for that category. These images were generated from simplified data, but are aligned to the center of the drawing's bounding box rather than top-left corner [4].

[Original Code snippet which was used for this preprocessing](#)

For the purpose of this project we are using 10000 images for the first 30 categories, 3,00,000 records in total.

2.1 Data Loading

To load the dataset, we have downloaded the .npy files and loaded them locally from our system as shown in Figure 2. We have set the `FILES_LEN = 30`, considering the images data for the first 30 classes. Iterating over the selected files, we chose the first 10,000 images for each class for which `IMAGES_PER_CLASS` is set to 10,000 initially. This would allow us to train the model and perform different experiments to expect a fairly high accuracy. [Cell-11]

```

files = os.listdir(dataset_path)[:FILES_LEN]
classes = []

all_images = []
all_labels = []
for index, file in enumerate(files):
    class_name = file.replace(' ', '_').replace('.npy', '')
    classes.append(class_name)
    data = np.load(os.path.join(dataset_path, file)).astype(np.float32)
    data = data[:IMAGES_PER_CLASS]

    # Normalize pixel values to [0, 1]
    data = data / 255.0
    data = data.reshape(-1, 28, 28)
    all_images.extend(data)
    all_labels.extend([index] * len(data))

all_images = np.array(all_images)
all_labels = np.array(all_labels)

print(f"Total images: {len(all_images)}, Total labels: {len(classes)}")

```

Total images: 300000, Total labels: 30

Figure 2: Code snippet used to load the dataset

2.2 Data Visualisation

For visualisation of some key attributes of the dataset, we have used some figures as shown below:

- Figure 3 shows some random doodle samples from the preprocessed dataset which shows the diverseness of the dataset. For example, for class “**angel**” people have used very different strokes and drawn completely different drawings, hence diversified dataset for model training. [Cell-13]

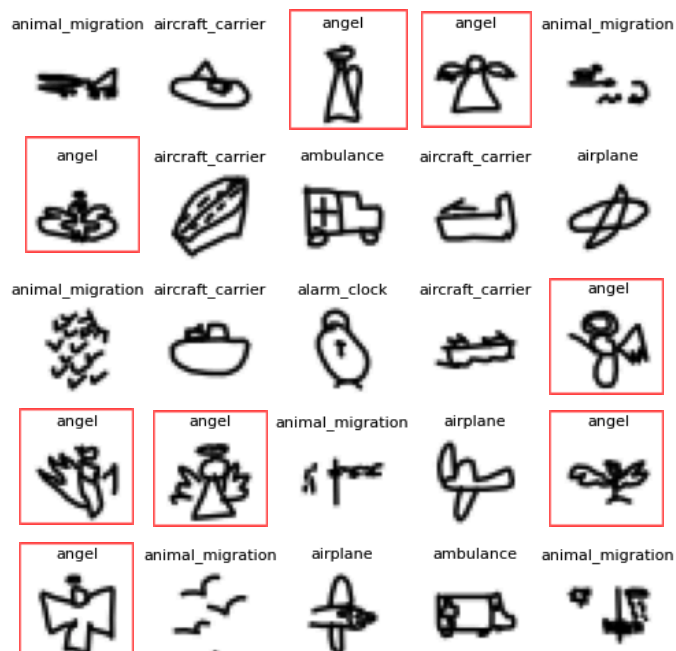


Figure 3: Visualised dataset samples after preprocessing

- *Figure 4*, shows the mean stroke distribution of a random sample image. All the figures are randomly generated; here the example shows a “**bed**” sample, and the average of stroke distribution of all the **bed** category images. The lighter area in the graph depicts the regions where the users have more likely drawn a stroke while doodling for this category, whereas the darker area represents the regions where the strokes are rarely drawn by any player. [Cell-15]

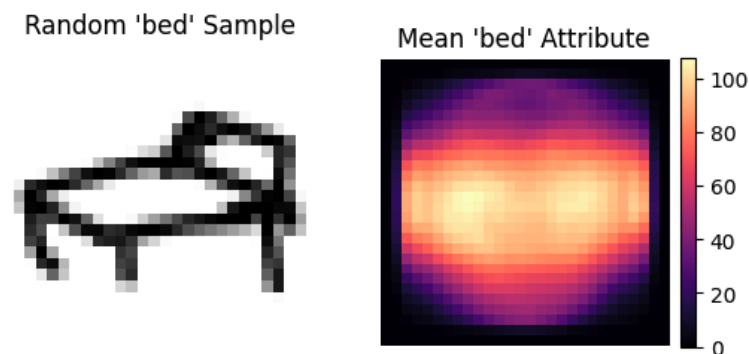


Figure 4: Mean stroke distribution of a sample doodle

- *Figure 5*, shows the sparsity distribution of 3 random doodle samples. It represents the percentage of non-zero pixels in the image to show how “busy” a sketch is. As shown in the figure, the “**basket**” has more percentage of non-zero pixels i.e, more ink or strokes are used to draw this as compared to other two doodles. [Cell-16]

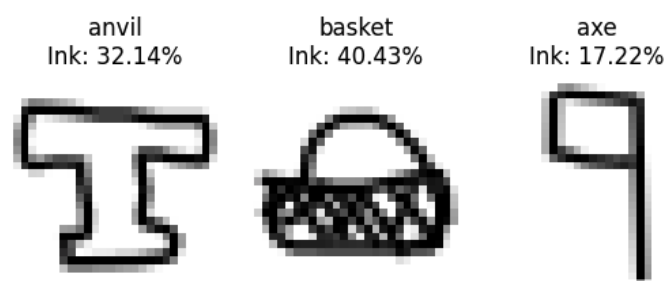


Figure 5: Sparsity distribution of 3 sample doodles

- *Figure 6*, displays a heatmap for 10 random categories out of the selected 30 from the dataset. It represents the correlation coefficients to specify how much two categories are likely to be similar to each other considering the sample images of those categories. For example, correlation coefficient values of “**bandage**” and “**airplane**” is **0.94**, which is very high; it explains that doodles of these categories are similar to a high extent. [Cell-17]

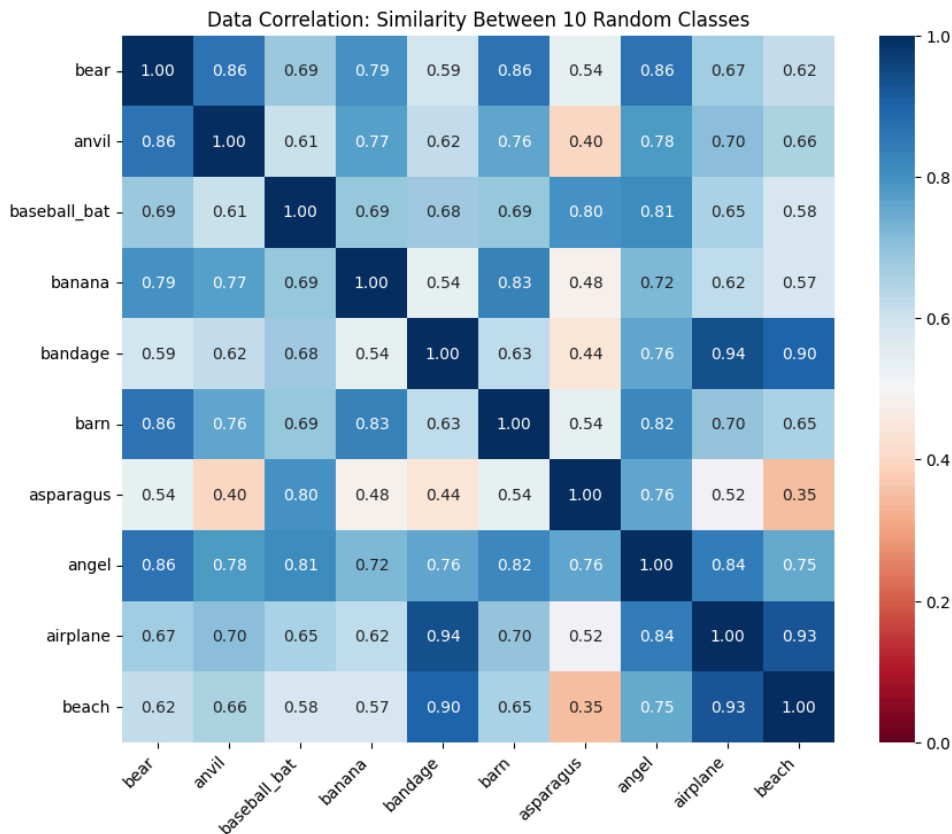


Figure 6: Correlation heatmap for 10 random categories

2.3 Data Pre Processing

We have used the pre processed simplified version of the dataset where the simplification process applied is as follows:

“

1. Align the drawing to the top-left corner, to have minimum values of 0.
2. Uniformly scale the drawing, to have a maximum value of 255.
3. Resample all the strokes with a 1 pixel spacing.
4. Simplify all strokes using the Ramer-Douglas-Peucker-algorithm[6] with an epsilon value of 2.0.

”[4]

We transformed these 28x28 simplified images to 56x56 and then normalized with a mean of 0.485 and standard deviation of 0.229. These values were picked from the pytorch documentation [9]. [Cell-21]

2.4 Data Split

We have used the `train_test_split()` method by scikit-learn to split the dataset into training, validation and test sets in the ratio 70:15:15 respectively. We believe that a validation set is necessary so that the model is updated after every epoch based on the validation set loss and we were also able to detect overfitting in the case of a pretrained model. [Cell-19]

3. Network Structure

3.1 Network Architecture

We have used ConvNeXt-tiny for the purpose of this project as it is a lightweight version. ConvNeXt is a tuned version of ResNet-50 and ResNet-200 [12] also shown in *Figure 7*. It follows an approach called depthwise convolution [12] which aims to reduce the computational complexity as low as possible and achieve Transformer-level performance or better than that. As shown in figure 7 and 8, the first convolutional layer consists of a kernel of size 4x4 and a stride of 4; it is a replacement of the 7x7 ResNet layer with maxpool which is also called the “patchify” system as it splits the image into a sequence of patches. Additionally, the stage compute ratio was also changed from (3,4,6,3) of ResNet to (3,3,9,3), simply saying the number of blocks were adjusted in each stage [2], also shown in *Figure 8* ConvNeXt architecture. The authors mentioned that it helped in improving model accuracy from 78.8 percent to 79.4 percent.

This is followed by a 4 block system where each block has a depthwise convolution of 7x7, layer normalisation, a 1x1 convolution to expand the output in 4x channels, GELU activation layer and 1x1 convolution for compression back to original channels, this is called the “*inverted bottleneck structure*”. Some micro level changes in ConvNeXt is that there is no application of activation and normalisation layer after every convolution, they are applied once per block. ReLU in ResNet is replaced with GELU(Gaussian Error Linear Unit) to achieve smoother gradients and Batch Normalisation is replaced with Layer Normalisation, which is applied at the start of the block and before the downsampling layers. All this is followed by downsampling layers using a 2x2 convolution with stride 2 to reduce spatial resolution whereas ResNet downsamples within a residual block. Downsampling is followed by an average pooling layer and a fully connected layer as a linear classifier.

The total number of trainable parameters in our project is approximately 28 Million.

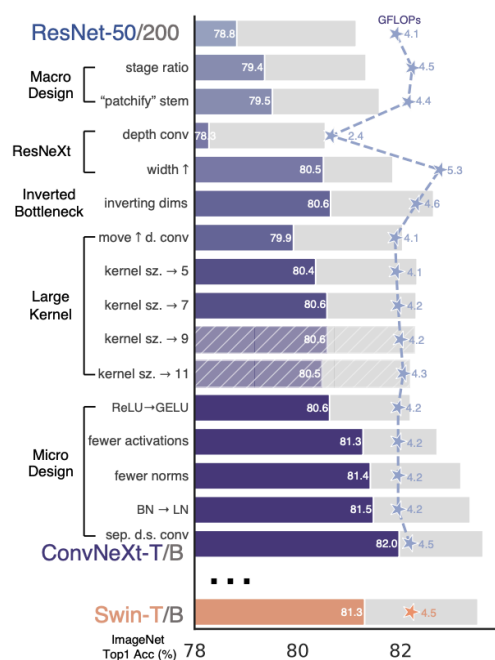


Figure 7: Modernization of ResNet to ConvNeXt [2]

	output size	• ResNet-50	• ConvNeXt-T	○ Swin-T
stem	56×56	7×7, 64, stride 2 3×3 max pool, stride 2	4×4, 96, stride 4	4×4, 96, stride 4
res2	56×56	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} d7 \times 7, 96 \\ 1 \times 1, 384 \\ 1 \times 1, 96 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 96 \times 3 \\ \text{MSA, } w7 \times 7, H=3, \text{ rel. pos.} \\ 1 \times 1, 96 \\ 1 \times 1, 384 \\ 1 \times 1, 96 \end{bmatrix} \times 2$
res3	28×28	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} d7 \times 7, 192 \\ 1 \times 1, 768 \\ 1 \times 1, 192 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 192 \times 3 \\ \text{MSA, } w7 \times 7, H=6, \text{ rel. pos.} \\ 1 \times 1, 192 \\ 1 \times 1, 768 \\ 1 \times 1, 192 \end{bmatrix} \times 2$
res4	14×14	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} d7 \times 7, 384 \\ 1 \times 1, 1536 \\ 1 \times 1, 384 \end{bmatrix} \times 9$	$\begin{bmatrix} 1 \times 1, 384 \times 3 \\ \text{MSA, } w7 \times 7, H=12, \text{ rel. pos.} \\ 1 \times 1, 384 \\ 1 \times 1, 1536 \\ 1 \times 1, 384 \end{bmatrix} \times 6$
res5	7×7	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} d7 \times 7, 768 \\ 1 \times 1, 3072 \\ 1 \times 1, 768 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 768 \times 3 \\ \text{MSA, } w7 \times 7, H=24, \text{ rel. pos.} \\ 1 \times 1, 768 \\ 1 \times 1, 3072 \\ 1 \times 1, 768 \end{bmatrix} \times 2$
FLOPs		4.1×10^9	4.5×10^9	4.5×10^9
# params.		25.6×10^6	28.6×10^6	28.3×10^6

Figure 8: The comparison of architecture of ResNet-50, ConvNeXt-T and Swin-T [12]

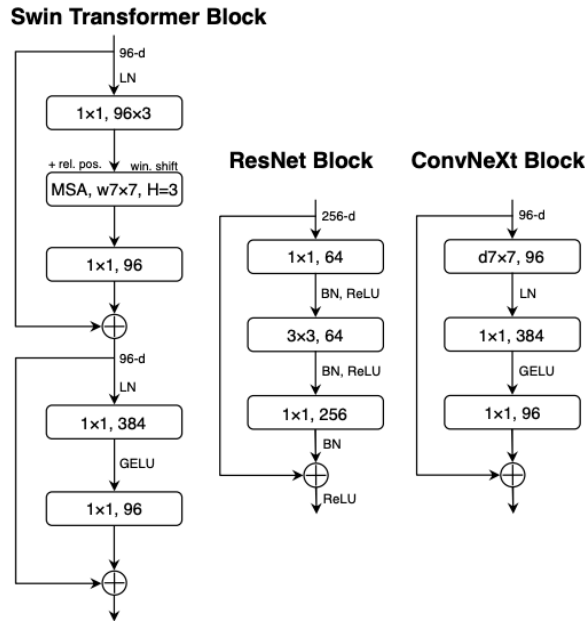


Figure 9: Block designs for Swin Transformers, ResNet and ConvNeXt [2]

We also have used a non-default version of ConvNeXt-tiny without any initialised weights, hence it is trained from scratch. As shown in the attached *Figure 10*, we have changed the number of input channels from 3 to 1 i.e., grayscale but the kernel size and stride remains the same as 4x4 and 4 respectively. We have also added a Sequential block with a dropout layer of probability 0.2 to prevent overfitting because the pretrained model depicted an evident overfitting after around 3 epochs. Dropout is followed by a linear layer to map input

features to the number of output classes which is 30 in our case. The remaining architecture was kept the same.

The total number of trainable parameters in this case remains the same, approximately 28 million. [Cell-36]

```
num_classes = len(classes)

# Load convnext tiny with random weights (no pretrained weights)
scratch_model = convnext_tiny(weights=None)
print("Loaded ConvNeXt Tiny with random weights")

# Changed the input from 3 to 1 channel for grayscale images
scratch_model.features[0][0] = nn.Conv2d(1, 96, kernel_size=4, stride=4)

# Modify the classifier head
in_features = scratch_model.classifier[2].in_features
scratch_model.classifier[2] = nn.Sequential([
    nn.Dropout(0.2),
    nn.Linear(in_features, num_classes)
])

scratch_model = scratch_model.to(device)

print(scratch_model)

total_params = sum(p.numel() for p in scratch_model.parameters())
trainable_params = sum(p.numel() for p in scratch_model.parameters() if p.requires_grad)
```

Figure 10: Code snippet we used to initialise the model trained from scratch

4. Loss function

Loss functions are used to calculate a model's performance on seen as well as unseen data. It calculates the difference between each predicted and expected sample or data point for an entire epoch. As the name suggests, it gives the loss value considering all the data points, and a lower value means that the model is well-trained. We have used the Cross-Entropy Loss Function. Multi-Class classification problems have Cross Entropy loss as a standard metric, and it heavily penalises incorrect outputs which is why we chose this. [Cell-32]

$$L = - \sum_{k=1}^K y_k \log(p_k)$$

Figure 11: Standard formula for cross entropy loss which is the negative sum of products of output label and the logarithm of probability of that output. (Photo taken from medium article [8])

5. Optimiser

We have used AdamW as the optimiser based on the training procedure of the original ConvNeXt. Optimisers are used while training a model so that it adjusts its weights and parameters during backpropagation to give better results and eventually minimise overall loss. We used a learning rate of 0.0005 for the pretrained model and 0.01 for the model trained from scratch. Both the models have a weight decay of 0.1 and use cosine decay. [Cell-13]

```
criterion = nn.CrossEntropyLoss()

# The reason for choosing adamW and cosine annealing was that the original convnext
# and they are known to work well for vision transformers, which is what convNext is
optimizer = optim.AdamW(pretrained_model.parameters(), lr=0.0005, weight_decay=0.1)
scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=EPOCHS)
```

✓ 0.0s

Python

Figure 12: Code snippet we used to implement loss function and AdamW optimiser

6. Experiments

6.1 Data Preprocessing

6.1.1 Input Channel/Size

By default ConvNext Tiny takes 3 channel 224x224 images as inputs. Initially (Mk4 Model) we upscaled our images and converted them to 3 channels, but it was causing memory issues during transformation, so we had to reduce the size to 1000 images per class. Later (Mk6 Model) we changed it to single channel and 56x56 input size, reducing the transform size so we could increase our image per class to 10000. This not only increased the test accuracy from **85.83%** to **87.44%** but also decreased the test loss from **0.7815** to **0.5938**.

Classification Report of Mk4 Model:

	precision	recall	f1-score	support
aircraft_carrier	0.5268	0.5900	0.5566	100
airplane	0.8947	0.8500	0.8718	100
alarm_clock	0.8854	0.8500	0.8673	100
ambulance	0.8942	0.9300	0.9118	100
angel	0.8990	0.8900	0.8945	100
animal_migration	0.6937	0.7700	0.7299	100
ant	0.8396	0.8900	0.8641	100
anvil	0.8846	0.9200	0.9020	100
apple	0.9596	0.9500	0.9548	100
arm	0.8800	0.8800	0.8800	100
asparagus	0.9247	0.8600	0.8912	100
axe	0.9681	0.9100	0.9381	100
backpack	0.9000	0.9000	0.9000	100
banana	0.9794	0.9500	0.9645	100

bandage	0.8600	0.8600	0.8600	100
barn	0.9100	0.9100	0.9100	100
baseball_bat	0.8500	0.8500	0.8500	100
baseball	0.8438	0.8100	0.8265	100
basket	0.8763	0.8500	0.8629	100
basketball	0.8515	0.8600	0.8557	100
bat	0.8889	0.8000	0.8421	100
bathtub	0.7981	0.8300	0.8137	100
beach	0.7963	0.8600	0.8269	100
bear	0.7870	0.8500	0.8173	100
beard	0.8725	0.8900	0.8812	100
bed	0.8235	0.8400	0.8317	100
bee	0.9167	0.8800	0.8980	100
belt	0.8690	0.7300	0.7935	100
bench	0.8100	0.8100	0.8100	100
bicycle	0.9608	0.9800	0.9703	100
accuracy			0.8583	3000
macro avg	0.8615	0.8583	0.8592	3000
weighted avg	0.8615	0.8583	0.8592	3000

Classification Report of Mk6 Model:

	precision	recall	f1-score	support
aircraft_carrier	0.6663	0.6070	0.6353	1000
airplane	0.8200	0.8790	0.8485	1000
alarm_clock	0.9153	0.8750	0.8947	1000
ambulance	0.9175	0.9570	0.9369	1000
angel	0.8609	0.9100	0.8848	1000
animal_migration	0.7563	0.8350	0.7937	1000
ant	0.9075	0.9030	0.9053	1000
anvil	0.8811	0.9340	0.9068	1000
apple	0.9488	0.9640	0.9563	1000
arm	0.8991	0.9000	0.8996	1000
asparagus	0.8593	0.8730	0.8661	1000
axe	0.9448	0.9250	0.9348	1000
backpack	0.8922	0.8860	0.8891	1000
banana	0.9594	0.9220	0.9403	1000
bandage	0.9298	0.8480	0.8870	1000
barn	0.8838	0.9130	0.8982	1000
baseball_bat	0.9020	0.8840	0.8929	1000
baseball	0.8037	0.8310	0.8171	1000
basket	0.8869	0.8470	0.8665	1000
basketball	0.8274	0.8150	0.8212	1000
bat	0.9167	0.7700	0.8370	1000
bathtub	0.8727	0.8570	0.8648	1000
beach	0.8724	0.8550	0.8636	1000
bear	0.8171	0.8620	0.8389	1000
beard	0.9097	0.8760	0.8925	1000
bed	0.8724	0.8550	0.8636	1000
bee	0.9084	0.9120	0.9102	1000
belt	0.8686	0.8460	0.8571	1000
bench	0.7979	0.9160	0.8529	1000

bicycle	0.9654	0.9760	0.9707	1000
accuracy			0.8744	30000
macro avg	0.8755	0.8744	0.8742	30000
weighted avg	0.8755	0.8744	0.8742	30000

6.1.2 Random Affine

Mk2 and Mk3 models both used 3 channels with 224x224 input but they differ in data augmentation. The Mk3 model uses affine transformations that can rotate, scale and translate the images. This change resulted in a decrease in accuracy from **86.57%** to **85.40%**.

Classification Report for Mk2 Model:

	precision	recall	f1-score	support
aircraft_carrier	0.5977	0.5200	0.5561	100
airplane	0.8544	0.8800	0.8670	100
alarm_clock	0.9091	0.9000	0.9045	100
ambulance	0.9029	0.9300	0.9163	100
angel	0.8889	0.8800	0.8844	100
animal_migration	0.7143	0.7500	0.7317	100
ant	0.9072	0.8800	0.8934	100
anvil	0.9184	0.9000	0.9091	100
apple	0.9588	0.9300	0.9442	100
arm	0.8364	0.9200	0.8762	100
asparagus	0.9053	0.8600	0.8821	100
axe	0.9479	0.9100	0.9286	100
backpack	0.9278	0.9000	0.9137	100
banana	0.9231	0.9600	0.9412	100
bandage	0.8763	0.8500	0.8629	100
barn	0.9100	0.9100	0.9100	100
baseball_bat	0.8660	0.8400	0.8528	100
baseball	0.8500	0.8500	0.8500	100
basket	0.9062	0.8700	0.8878	100
basketball	0.9000	0.8100	0.8526	100
bat	0.8542	0.8200	0.8367	100
bathtub	0.8411	0.9000	0.8696	100
beach	0.7586	0.8800	0.8148	100
bear	0.8182	0.9000	0.8571	100
beard	0.8738	0.9000	0.8867	100
bed	0.8200	0.8200	0.8200	100
bee	0.9255	0.8700	0.8969	100
belt	0.8351	0.8100	0.8223	100
bench	0.8235	0.8400	0.8317	100
bicycle	0.9333	0.9800	0.9561	100
accuracy			0.8657	3000
macro avg	0.8661	0.8657	0.8652	3000
weighted avg	0.8661	0.8657	0.8652	3000

Classification Report for Mk3 model:

	precision	recall	f1-score	support
aircraft_carrier	0.6180	0.5500	0.5820	100
airplane	0.8365	0.8700	0.8529	100
alarm_clock	0.8763	0.8500	0.8629	100
ambulance	0.9000	0.9000	0.9000	100
angel	0.9255	0.8700	0.8969	100
animal_migration	0.7604	0.7300	0.7449	100
ant	0.8725	0.8900	0.8812	100
anvil	0.8738	0.9000	0.8867	100
apple	0.9300	0.9300	0.9300	100
arm	0.8462	0.8800	0.8627	100
asparagus	0.8529	0.8700	0.8614	100
axe	0.9670	0.8800	0.9215	100
backpack	0.9072	0.8800	0.8934	100
banana	0.9495	0.9400	0.9447	100
bandage	0.8750	0.8400	0.8571	100
barn	0.9082	0.8900	0.8990	100
baseball_bat	0.8438	0.8100	0.8265	100
baseball	0.8333	0.8000	0.8163	100
basket	0.8878	0.8700	0.8788	100
basketball	0.8367	0.8200	0.8283	100
bat	0.6693	0.8500	0.7489	100
bathtub	0.8447	0.8700	0.8571	100
beach	0.7807	0.8900	0.8318	100
bear	0.8646	0.8300	0.8469	100
beard	0.8738	0.9000	0.8867	100
bed	0.8137	0.8300	0.8218	100
bee	0.9062	0.8700	0.8878	100
belt	0.8421	0.8000	0.8205	100
bench	0.8384	0.8300	0.8342	100
bicycle	0.9333	0.9800	0.9561	100
accuracy			0.8540	3000
macro avg	0.8556	0.8540	0.8540	3000
weighted avg	0.8556	0.8540	0.8540	3000

6.2 Layer Freezing

Since we are using pretrained models there were signs of overfitting. To reduce that, we tried freezing the layers. The Mk3 and Mk4 models have all the same parameters except the layers, Mk4 has the first 3 (0,1,2) layers frozen whereas Mk3 none. This resulted in a very minimal increase on accuracy from 85.40% to 85.83% on the test set, but interestingly overfitting increased, meaning the first 3 layers were actually learning. The only good thing that came out of this experiment was the reduction in training speed, which was expected. :(

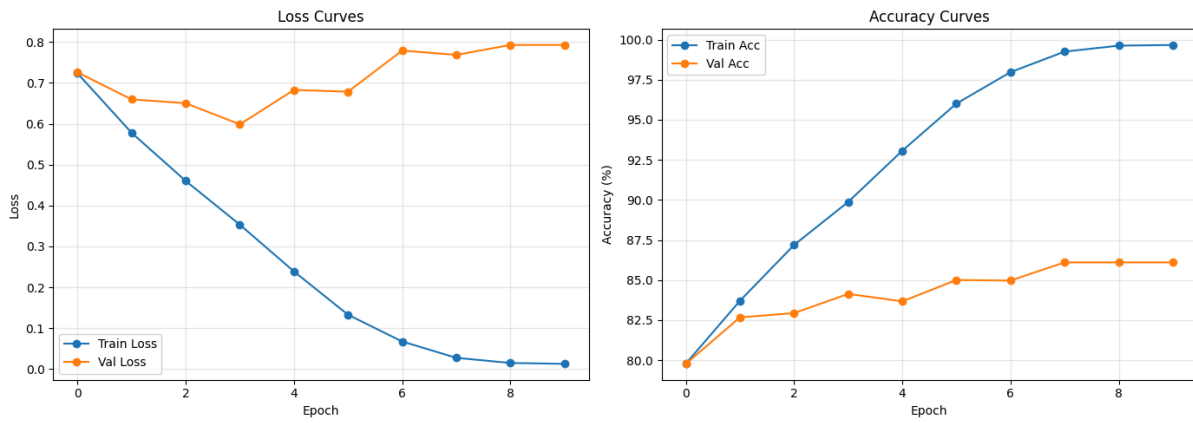


Figure 13: Mk3 training accuracy and loss curves

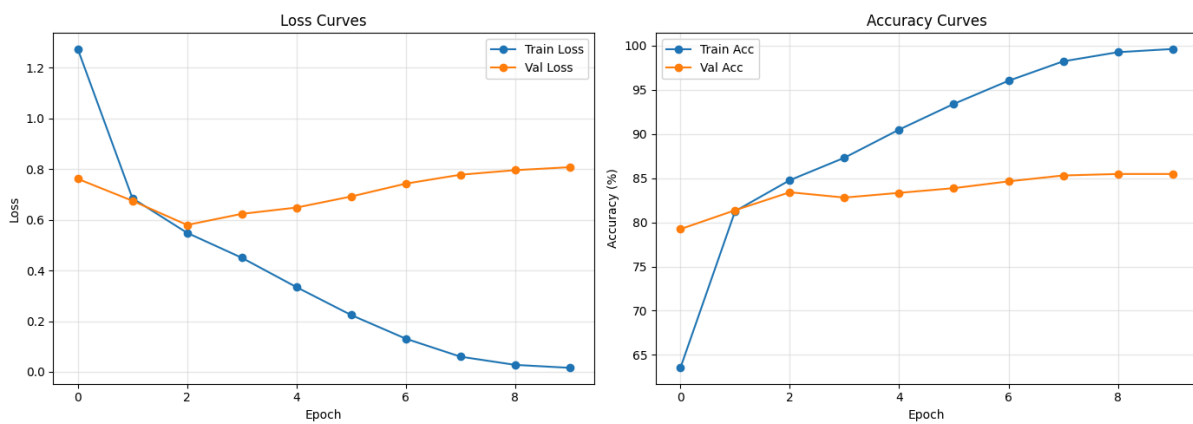


Figure 14: Mk4 training accuracy and loss curves

6.3 Learning Rate

The Mk6 model and Mk7 differ in learning rates and weight decay. Mk6 has a learning rate of 0.001 with a weight decay of 0.05, whereas Mk7 has a rate of 0.0005 with a decay of 0.1. This experiment was primarily done to reduce overfitting. As seen in the figures, though the loss decreased and the accuracy increased (from **87.66%** to **88.06%** on validation set), overfitting was still happening. This experiment basically reduced overfitting but didn't eliminate it. :(

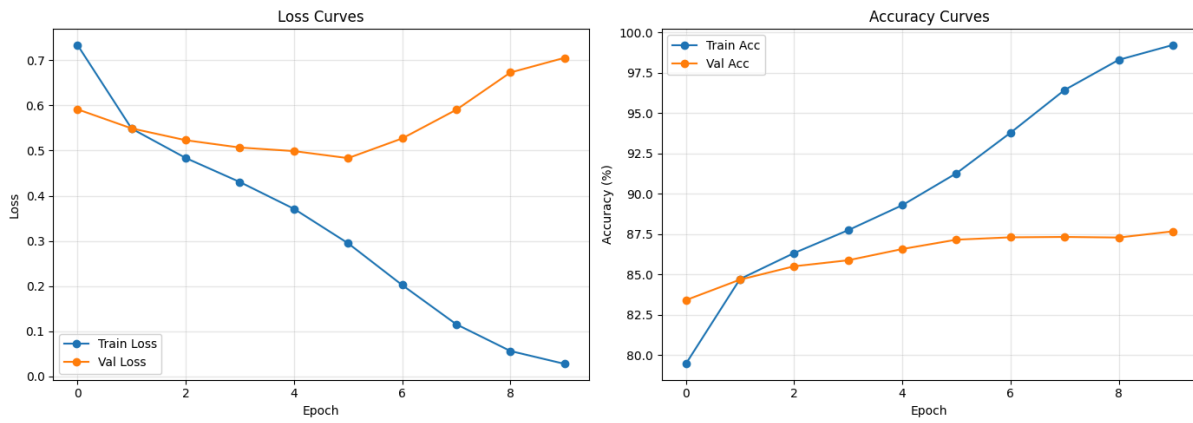


Figure 15: Mk6 training accuracy and loss curves

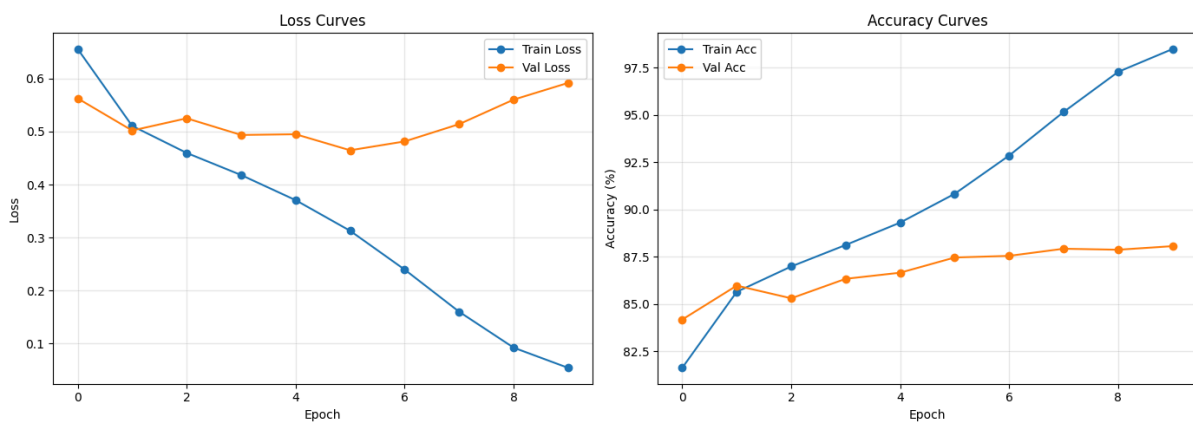


Figure 16: Mk7 training accuracy and loss curves.

6.4 Dropout

In order to decrease overfitting further we tried adding a dropout rate of 0.2 in the Mk8 model as compared to Mk7 model which had no configured dropout rate. This just resulted in an accuracy drop of 0.03, with still signs of overfitting.

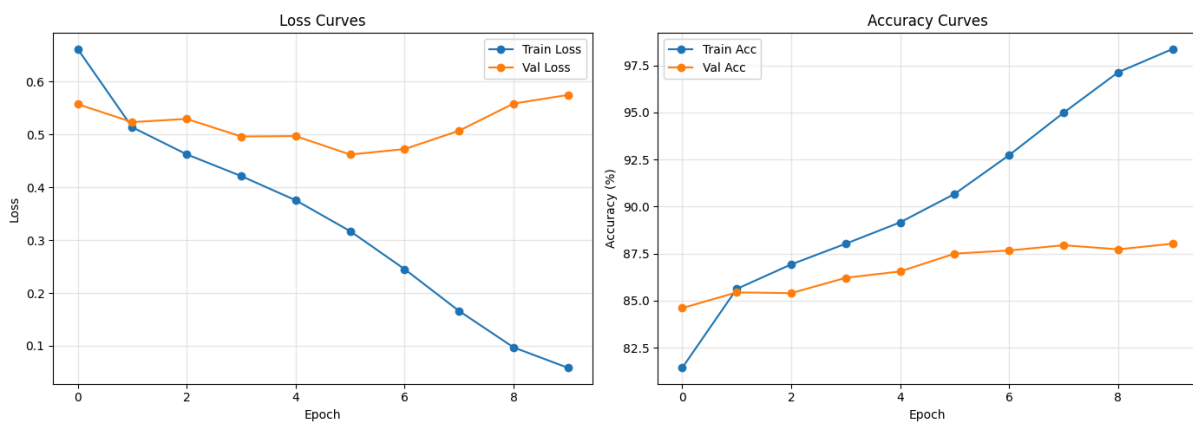


Figure 17: Mk8 training accuracy and loss curves

6.5 Training from Scratch

In the initial experiments our goal was to increase the accuracy, which was successful, but in the later experiments we tried to reduce overfitting, which was successful only to some extent. Then it occurred to us, what if we train the model from scratch, i.e use random weights instead of pretrained ImageNet1k weights [10]. The Mk9 model used randomly initialised weights. The results were quite impressive, there were no signs of overfitting :) but a drop in accuracy which was expected since we only trained it for 10 epochs.

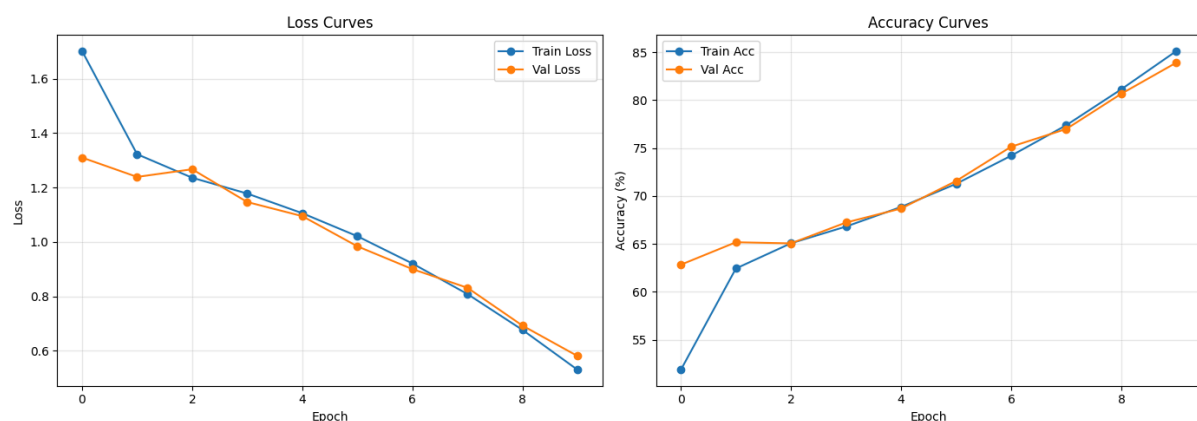


Figure 18: Mk9 training accuracy and loss curves

7. Results

After 9 experiments we created the final colab file that compares the pretrained model with the one trained from scratch. One change here from the previous experiments was the data split. So all 9 experiments have a split of 80:10:10 but the final one has 70:15:15 as mentioned in the dataset section above.

7.1 Pretrained Model

The pretrained model uses ImageNet1k weights [10] and has a learning of 0.0005 with a weight decay of 0.1. The model performed really well on the test dataset with an accuracy of 87.88% but showed clear signs of overfitting as shown in the learning curve below.

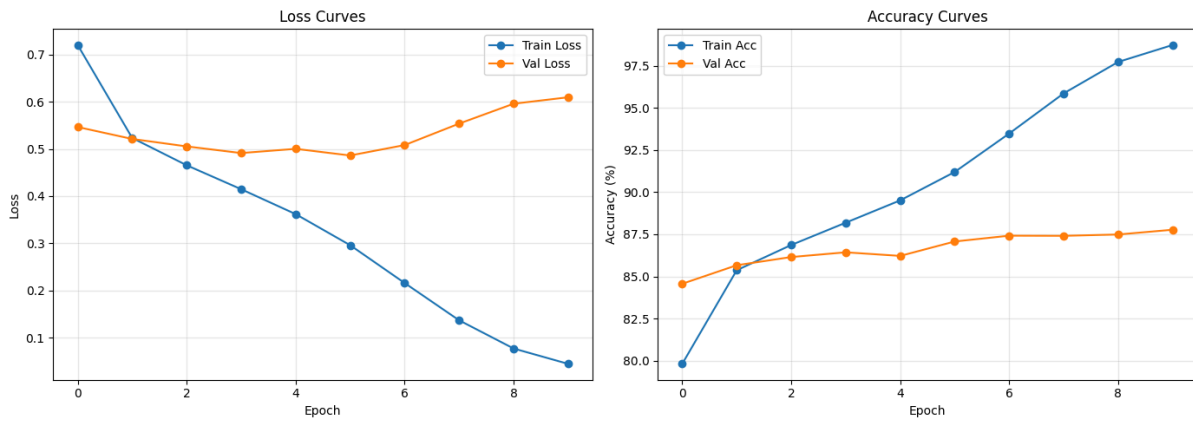


Figure 19: Pre-Trained model training accuracy and loss curves

The following classification report shows the accuracy metrics on the test dataset:

	precision	recall	f1-score	support
aircraft_carrier	0.6616	0.6060	0.6326	1500
airplane	0.8109	0.8720	0.8403	1500
alarm_clock	0.9051	0.9220	0.9135	1500
ambulance	0.9510	0.9453	0.9482	1500
angel	0.8998	0.8920	0.8959	1500
animal_migration	0.8252	0.7960	0.8103	1500
ant	0.8781	0.9173	0.8973	1500
anvil	0.9414	0.9107	0.9258	1500
apple	0.9692	0.9647	0.9669	1500
arm	0.9175	0.8900	0.9036	1500
asparagus	0.8694	0.8613	0.8654	1500
axe	0.9462	0.9267	0.9363	1500
backpack	0.8889	0.9013	0.8951	1500
banana	0.9134	0.9493	0.9310	1500
bandage	0.8713	0.8533	0.8622	1500
barn	0.9249	0.9113	0.9181	1500
baseball_bat	0.8616	0.9133	0.8867	1500
baseball	0.8264	0.8060	0.8161	1500
basket	0.9006	0.8700	0.8850	1500
basketball	0.8246	0.8433	0.8339	1500
bat	0.8329	0.8473	0.8401	1500
bathtub	0.8623	0.8640	0.8631	1500
beach	0.8512	0.8427	0.8469	1500
bear	0.8539	0.8687	0.8612	1500
beard	0.9052	0.8913	0.8982	1500
bed	0.8749	0.8860	0.8804	1500
bee	0.8973	0.9147	0.9059	1500
belt	0.8487	0.8487	0.8487	1500
bench	0.8731	0.8667	0.8699	1500
bicycle	0.9685	0.9827	0.9755	1500
accuracy			0.8788	45000
macro avg	0.8785	0.8788	0.8785	45000

weighted avg 0.8785 0.8788 0.8785 45000

7.2 Model Trained from Scratch

This model has randomly initialised weights but a higher learning rate and weight decay of 0.01 and 0.1 respectively. The results as expected were impressive, though there was a drop in accuracy, there were no signs of overfitting. The model generalised the data really well, with an accuracy of **83.84%** on the test set.

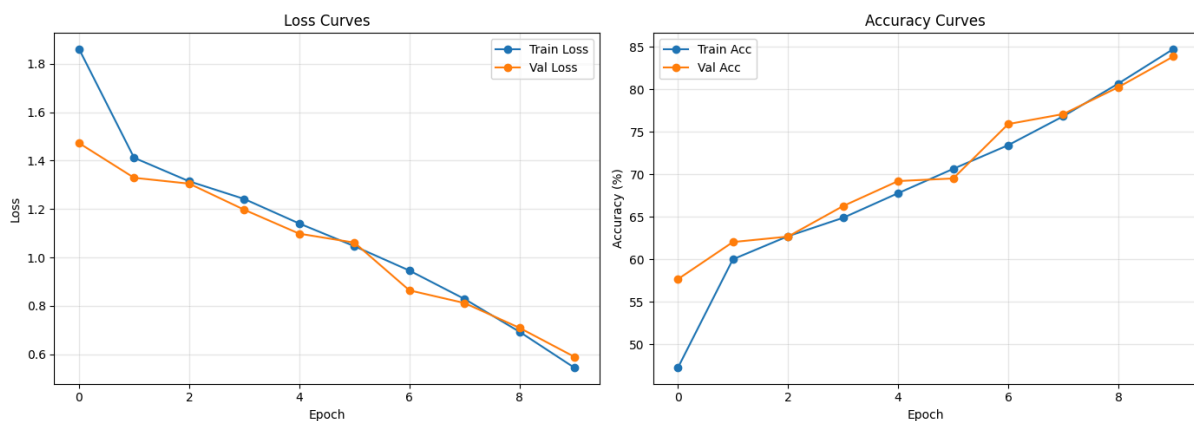


Figure 20: Random Weights (Scratch) model's training accuracy and loss curves

The following classification report shows the accuracy metrics on the test dataset:

	precision	recall	f1-score	support
aircraft_carrier	0.5899	0.5227	0.5543	1500
airplane	0.8149	0.8540	0.8340	1500
alarm_clock	0.8496	0.8660	0.8577	1500
ambulance	0.9097	0.9273	0.9185	1500
angel	0.8820	0.8520	0.8667	1500
animal_migration	0.6756	0.7927	0.7294	1500
ant	0.8828	0.8333	0.8573	1500
anvil	0.9301	0.9047	0.9172	1500
apple	0.9392	0.9573	0.9482	1500
arm	0.8348	0.8420	0.8384	1500
asparagus	0.7964	0.8607	0.8273	1500
axe	0.8951	0.8987	0.8969	1500
backpack	0.8655	0.8580	0.8617	1500
banana	0.9149	0.9313	0.9230	1500
bandage	0.8941	0.8220	0.8565	1500
barn	0.9115	0.8653	0.8878	1500
baseball_bat	0.8376	0.8907	0.8633	1500
baseball	0.8557	0.7273	0.7863	1500
basket	0.7915	0.8273	0.8090	1500
basketball	0.7727	0.8407	0.8052	1500

bat	0.8239	0.7640	0.7928	1500
bathtub	0.8520	0.8213	0.8364	1500
beach	0.8144	0.7400	0.7754	1500
bear	0.7398	0.7960	0.7669	1500
beard	0.8984	0.8133	0.8537	1500
bed	0.8601	0.8360	0.8479	1500
bee	0.7755	0.8867	0.8274	1500
belt	0.8094	0.8013	0.8054	1500
bench	0.8353	0.8520	0.8436	1500
bicycle	0.9403	0.9660	0.9530	1500
accuracy			0.8384	45000
macro avg	0.8397	0.8384	0.8380	45000
weighted avg	0.8397	0.8384	0.8380	45000

Model	Test Accuracy	Class on which best precision is achieved	Overfitting	Test Loss
ConvNeXt - Pretrained	87.88%	Apple (96.92%)	Yes after epoch 3	0.6097
ConvNeXt - Trained from Scratch	83.84%	Bicycle (94.03%)	No	0.5909

8. Suggested Improvements

While in our current implementation, we successfully applied ConvNeXt model to QuickDraw dataset which gives us the results we expected. We were able to eliminate overfitting done by the pretrained ConvNeXt model by training another version of ConvNeXt from scratch and we achieved much better results under the number of epochs. However, hardware limitations stopped us from further experimenting and investigating and we have a number of improvements that we look forward to implement:

- Train the scratch version of ConvNeXt on a larger subset of the dataset. Including more categories instead of 30 and more records per category might give better or interesting results.
- Use K-fold cross validation to train and validate on different folds of the dataset. Currently we are only splitting the dataset into training, validation and test set.
- Experimenting on more epochs would give a better trend of the loss curve and accuracy as well.

9. Related Work

ConvNeXt has been trained and pretrained on ImageNet-1K and ImageNet-22K dataset respectively to come up with a better performance compared with Vision Transformers. It is a pure CNN which uses 7x7 kernels and Layer Normalisation instead of Batch Normalisation. It was built based on the concept of ResNet and to match the accuracy of Transformer based models. [2]

On the other hand, Google's QuickDraw dataset is also used in numerous projects by people but one of the highlighted ones is training the Sketch-RNN [11]. In the experiment, Sketch-RNN was based on a sequence-to-sequence auto-encoder framework which helped to encode an input sequence into a vector of floating point numbers while training and achieve replication of input sequence in an output sequence using a decoder as closely as possible [13]. The .npz compressed files have been used which are suitable to train an RNN and a total 75,000 samples were used in the ratio 93 : 3.5 : 3.5 as training, validation and test set respectively. The experiment presented a neural network where it was trained on hand drawn doodles and the aim was to make it capable of producing sketches of common objects. In the experiment, the contributors were able to achieve the model producing weird images of animals with some mistakes in the body parts.

10. Statement of Work

We both worked on the same colab file where both of us iteratively developed and compared multiple model architectures and their results. We also worked together on parts of the colab notebook, report and the model architecture to come up with a final model which doesn't have overfitting as seen in our tested experiments.

Jai: I handled the selection for CNN model and based on the chosen QuickDraw dataset, I figured out that ConvNeXt goes really well with it as it is a simple model complementing our complex dataset. Also, I did not find any ConvNeXt implementation on this dataset so we decided to try something new. I also handled the transformations done on the preprocessed dataset, and plotted the results for different experiments to understand everything better. I also managed the local training environment to execute experimental code because of Simran's hardware limitations.

Simran: I am a regular player of the QuickDraw doodling game and while searching for an image dataset, I decided that working on this dataset would be really interesting. I also managed Exploratory Data Analysis (EDA) and figured out different types of visualisation, which can give some interesting insights on the dataset. I also implemented a train/test/validation split on the dataset. I also tried implementing k-fold cross validation, but due to hardware restrictions I wasn't successful there. I focused on documenting everything properly on the report while Jai executed the experiments.

11. AI Prompts Used

1. What all metrics can I add in the classification report?
2. How to add AUC metric, given this is my metric evaluation code?
3. Full code for implementing dataset streaming due to ram limitations during transform. [This failed, and later we downscaled the image instead of complicating things.]
4. Be Blunt, do you think overfitting is happening here? [Training logs were attached with the prompt]
5. Why does transform take so long? I mean 3 mins and only 33% My total training images are 276,000
6. Explain in detail the cosine annealing and its usage in convNext.
7. What is the difference between Adam and AdamW?
8. Okay so I have a 6gb graphics card Nvidia 3050 and this is my processor Processor AMD Ryzen 9 5900HS with Radeon Graphics, 3301 Mhz, 8 Core(s), 16 Logical Processor(s) with 32 GB of ram what should be my batch size and where should I train the convNext model?
9. If I have a QuickDraw dataset, and I am using .npy files for different classes, what visualisation techniques are there which are suitable for this kind of dataset?

12. Resources

- [1] <https://docs.pytorch.org/vision/stable/models/ConvNeXt.html>
- [2] <https://arxiv.org/abs/2201.03545>
- [3] https://huggingface.co/docs/transformers/model_doc/ConvNeXt
- [4] <https://github.com/googlecreativelab/quickdraw-dataset>
- [5] <https://quickdraw.withgoogle.com/data>
- [6] https://en.wikipedia.org/wiki/Ramer%E2%80%93Douglas%E2%80%93Peucker_algorithm
- [7] <https://medium.com/@venkateshmungara/ConvNeXt-a-family-of-pure-convnet-models-aff07d266c29>
- [8] <https://medium.com/@chris.p.hughes10/a-brief-overview-of-cross-entropy-loss-523aa56b75d5>
- [9] <https://docs.pytorch.org/vision/stable/transforms.html#torchvision.transforms.Normalize>
- [10] https://docs.pytorch.org/vision/stable/models/generated/torchvision.models.convnext_tiny.html#torchvision.models.ConvNeXt_Tiny_Weights
- [11] <https://arxiv.org/abs/1704.03477>
- [12] <https://towardsdatascience.com/the-cnn-that-challenges-vit/>
- [13] <https://research.google/blog/teaching-machines-to-draw/>